

Graphs

Chapter 10

- With Question/Answer Animations

10.3.5 Isomorphism of Graphs 图同构

Definition: The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there is a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 , for all a and b in V_1 . Such a function f is called an *isomorphism*. Two simple graphs that are not isomorphic are called *nonisomorphic*.

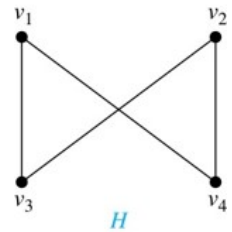
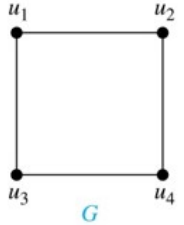
A function f is a *one-to-one correspondence*(一一对应的), or a *bijection*(双射函数), if it is both one-to-one and onto (surjective and injective).

Isomorphism of Graphs₂

Example: Show that the graphs $G = (V, E)$ and $H = (W, F)$ are isomorphic.

Solution: The function f with $f(u_1) = v_1$, $f(u_2) = v_4$, $f(u_3) = v_3$, and $f(u_4) = v_2$ is a

one-to-one correspondence between V and W . Note that adjacent vertices in G are u_1 and u_2 , u_1 and u_3 , u_2 and u_4 , and u_3 and u_4 . Each of the pairs $f(u_1) = v_1$ and $f(u_2) = v_4$, $f(u_1) = v_1$ and $f(u_3) = v_3$, $f(u_2) = v_4$ and $f(u_4) = v_2$, and $f(u_3) = v_3$ and $f(u_4) = v_2$ consists of two adjacent vertices in H .



10.3.6 Determining whether Two Simple Graphs are Isomorphic

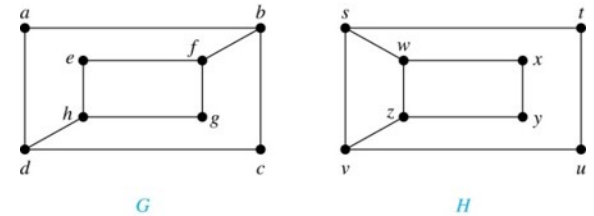
It is difficult to determine whether two simple graphs are isomorphic using brute force because there are $n!$ possible one-to-one correspondences between the vertex sets of two simple graphs with n vertices.

Sometimes it is not hard to show that two graphs are not isomorphic. We can do so by finding a property, preserved 保留 by isomorphism, that only one of the two graphs has. Such a property is called *graph invariant*. 图常量

There are many different useful graph invariants that can be used to distinguish nonisomorphic graphs, such as the number of vertices, number of edges, and degree sequence (list of the degrees of the vertices in nonincreasing order).

Isomorphism of Graphs₄

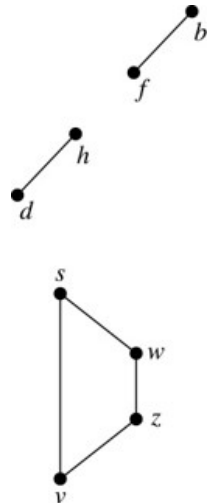
Example: Determine whether these two graphs are isomorphic.



Solution: Both graphs have eight vertices and ten edges. They also both have four vertices of degree two and four of degree three.

However, G and H are not isomorphic. Note that since $\deg(a) = 2$ in G , a must correspond to t , u , x , or y in H , because these are the vertices of degree 2. But each of these vertices is adjacent to another vertex of degree two in H , which is not true for a in G .

Alternatively, note that the subgraphs of G and H made up of vertices of degree three and the edges connecting them must be isomorphic. But the subgraphs, as shown at the right, are not isomorphic.



[Jump to long description](#)

Isomorphism of Graphs₅

Example: Determine whether these two graphs are isomorphic.

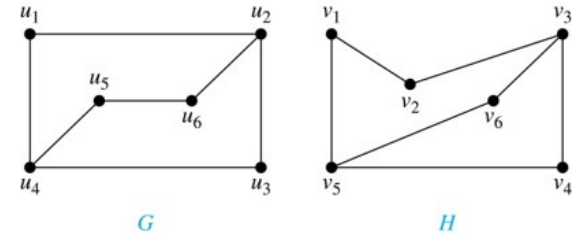
Solution: Both graphs have six vertices and seven edges.

They also both have four vertices of degree two and two of degree three.

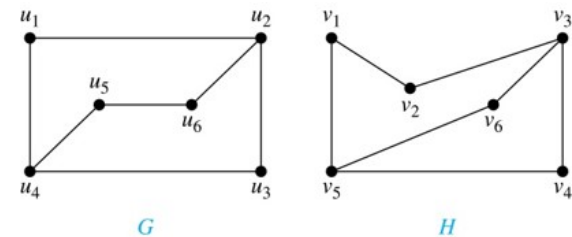
The subgraphs of G and H consisting of all the vertices of degree two and the edges connecting them are isomorphic. So, it is reasonable to try to find an isomorphism f .

We define an injection f from the vertices of G to the vertices of H that **preserves the degree of vertices**. We will determine whether it is an isomorphism.

The function f with $f(u_1) = v_6$, $f(u_2) = v_3$, $f(u_3) = v_4$, and $f(u_4) = v_5$, $f(u_5) = v_1$, and $f(u_6) = v_2$ is a one-to-one correspondence between G and H . **To see whether f preserves edges, we examine the adjacency matrix of G ,**



$$\mathbf{A}_G = \begin{matrix} & \begin{matrix} u_1 & u_2 & u_3 & u_4 & u_5 & u_6 \end{matrix} \\ \begin{matrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix},$$



and the adjacency matrix of H with the rows and columns labeled by the images of the corresponding vertices in G ,

$$\mathbf{A}_H = \begin{matrix} & \begin{matrix} v_6 & v_3 & v_4 & v_5 & v_1 & v_2 \end{matrix} \\ \begin{matrix} v_6 \\ v_3 \\ v_4 \\ v_5 \\ v_1 \\ v_2 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix}.$$

The function f with
 $f(u_1) = v_6$,
 $f(u_2) = v_3$,
 $f(u_3) = v_4$, and
 $f(u_4) = v_5$,
 $f(u_5) = v_1$, and
 $f(u_6) = v_2$

Because $A_G = A_H$, it follows that f preserves edges. We conclude that f is an isomorphism, so G and H are isomorphic. Note that if f turned out not to be an isomorphism, we would *not* have established that G and H are not isomorphic, because another correspondence of the vertices in G and H may be an isomorphism.

Section Summary₃

1, Adjacency Lists

Definition: An *adjacency list* 邻接表 can be used to represent a graph with no multiple edges by specifying the vertices that are adjacent to each vertex of the graph.

2, Adjacency Matrices

If the graph's adjacency matrix is $\mathbf{A}_G = [a_{ij}]$, then $a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$

3, Incidence Matrices

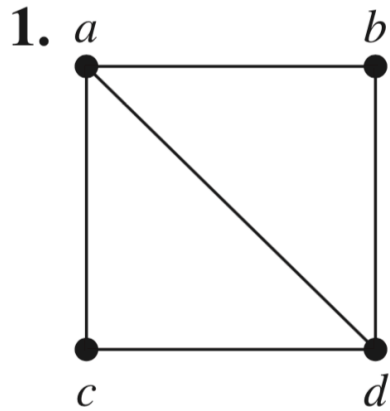
The *incidence matrix* 关联矩阵 with respect to the ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where $m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$

4, Isomorphism of Graphs

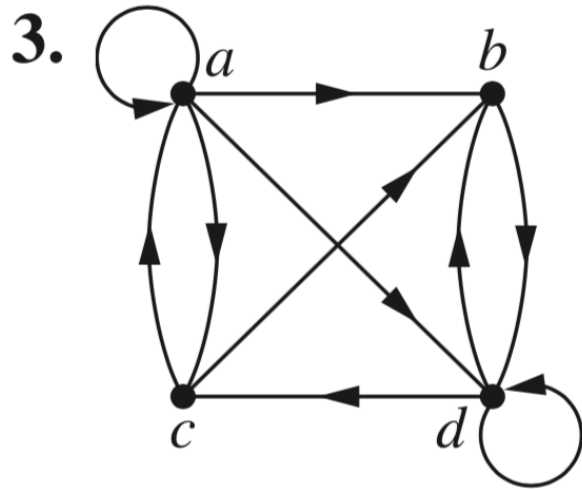
Definition: The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there is a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 , for all a and b in V_1 .

There are many different useful graph invariants that can be used to distinguish nonisomorphic graphs, such as the number of vertices, number of edges, and degree sequence.

Represent the graph in Exercise 1 with an adjacency list, an adjacency matrix, an incidence matrix.



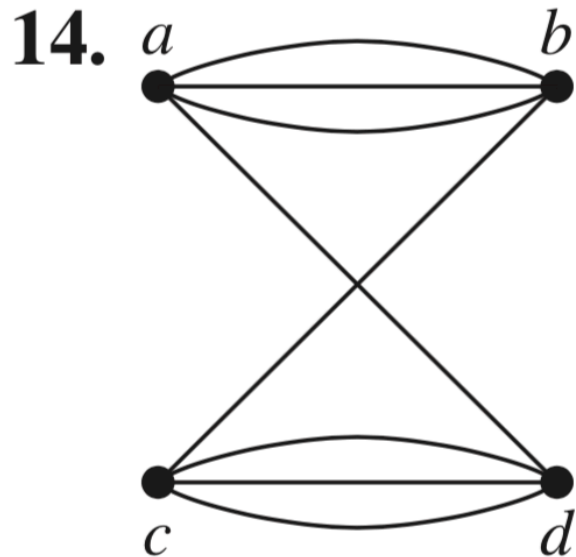
Represent the graph in Exercise 3 with an adjacency list, an adjacency matrix.



In Exercises 10–12 draw a graph with the given adjacency matrix.

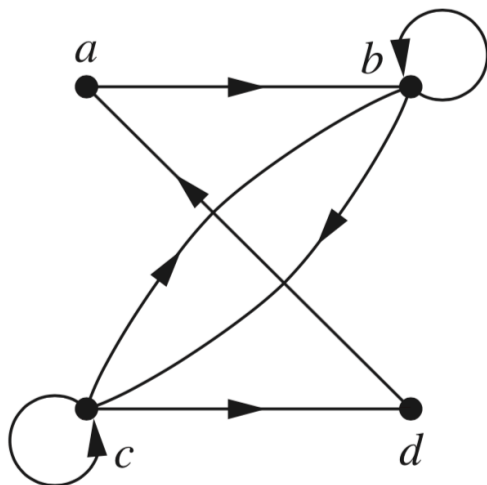
12.
$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

In Exercises 13–15 represent the given graph using an adjacency matrix.



In Exercises 19–21 find the adjacency matrix of the given directed multigraph with respect to the vertices listed in alphabetic order.

19.



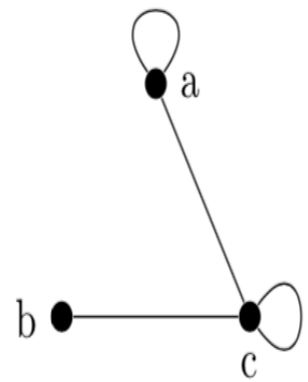
19.
$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

In Exercises 22–24 draw the graph represented by the given adjacency matrix.

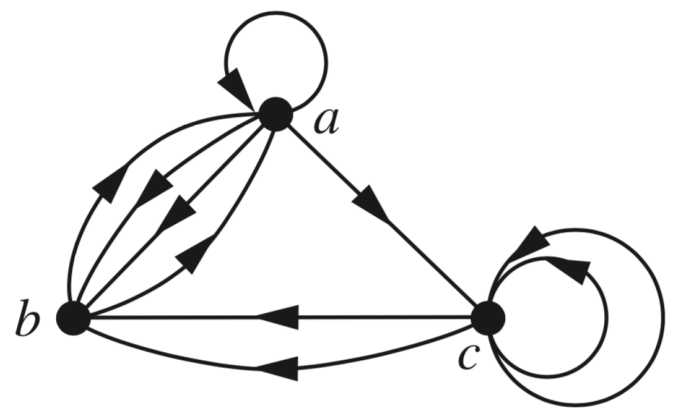
22.
$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

23.
$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 0 \\ 0 & 2 & 2 \end{bmatrix}$$

22. a) This matrix is symmetric, so we can take the graph to be undirected. No parallel edges are present, since no entries exceed 1.



23.



Homework

Page 683 Determine the type of graph (shown in Table 1) in exercise 3 5 7.

Page 699 2 7 22 35a

Draw these graphs K_5 $K_{2,5}$ C_5 W_5

Page 710 Represent the graph in Exercise 2 with an adjacency list, an adjacency matrix, an incidence matrix.

Page 710 11 15

Connectivity

Section 10.4

10.4.2 Paths₁

Informal Definition: A *path* 路径 is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex *along edges* of the graph. As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these edges.

Applications: Numerous problems can be modeled with paths formed by traveling along edges of graphs such as:

- determining whether a message can be sent between two computers.
- efficiently planning routes for mail delivery.

Paths₂

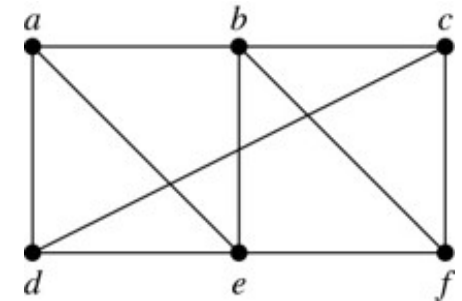
Definition: Let n be a nonnegative integer and G an undirected graph. A path of length n from u to v in G is a sequence of n edges $e_1, \dots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \dots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \dots, n$, the endpoints x_{i-1} and x_i .

- When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \dots, x_n$ (since listing the vertices uniquely determines the path).
- The path is a circuit 回路 if it begins and ends at the same vertex ($u = v$) and has length greater than zero.
- The path or circuit is said to pass through the vertices $x_1, x_2, \dots, x_{n-1}$ and traverse 遍历 the edges $e_1, \dots, e_n$.
- A path or circuit is simple if it does not contain the same edge more than once.

This terminology is readily extended to directed graphs. (see text)

Paths₃

Example: In the simple graph here:

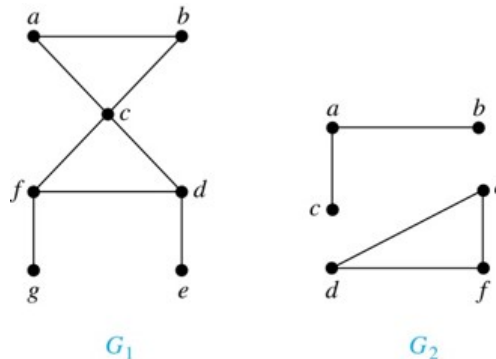


- a, d, c, f, e is a simple path of length 4.
- d, e, c, a is not a path because e is not connected to c .
- b, c, f, e, b is a circuit of length 4.
- a, b, e, d, a, b is a path of length 5, but it is not a simple path.

10.4.3 Connectedness in Undirected Graphs

Definition: An undirected graph is called *connected* 连通的 if there is a path between every pair of vertices. An undirected graph that is not *connected* is called *disconnected*. We say that we *disconnect* a graph when we remove vertices or edges, or both, to produce a disconnected subgraph.

Example: G_1 is connected because there is a path between any pair of its vertices, as can be easily seen. However G_2 is not connected because there is no path between vertices a and f , for example.

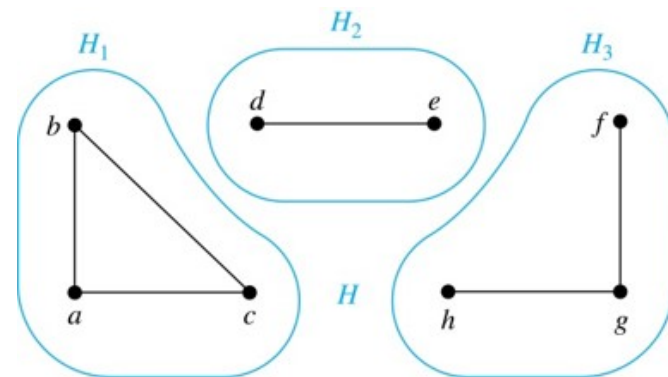


[Jump to long description](#)

Connected Components(opt)

Definition: A *connected component* 联通分支 of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G . A graph G that is not connected has two or more connected components that are disjoint and have G as their union.

Example: The graph H is the union of three disjoint subgraphs H_1 , H_2 , and H_3 , none of which are proper subgraphs of a larger connected subgraph of G . These three subgraphs are the connected components of H .



[Jump to long description](#)

10.4.5 Connectedness in Directed Graphs₁

Definition: A directed graph is *strongly connected* if there is a path from a to b and a path from b to a whenever a and b are vertices in the graph.

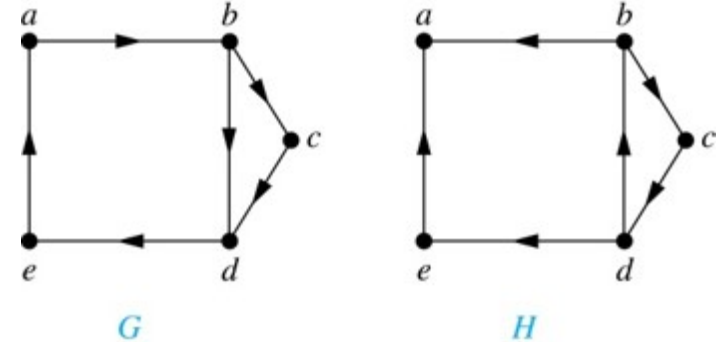
Definition: A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph, which is the undirected graph obtained by ignoring the directions of the edges of the directed graph.

Connectedness in Directed Graphs₂

Example: G is strongly connected because there is a path between any two vertices in the directed graph.

Hence, G is also weakly connected.

The graph H is not strongly connected, since there is no directed path from a to b , but it is weakly connected.



Definition: The subgraphs of a directed graph G that are strongly connected but not contained in larger strongly connected subgraphs, that is, the maximal strongly connected subgraphs, are called the *strongly connected components* 强联通分支 or *strong components* of G .

Example (continued): The graph H has three strongly connected components, consisting of the vertex a ; the vertex e ; and the subgraph consisting of the vertices b, c, d and edges (b, c) , (c, d) , and (d, b) .

[Jump to long description](#)

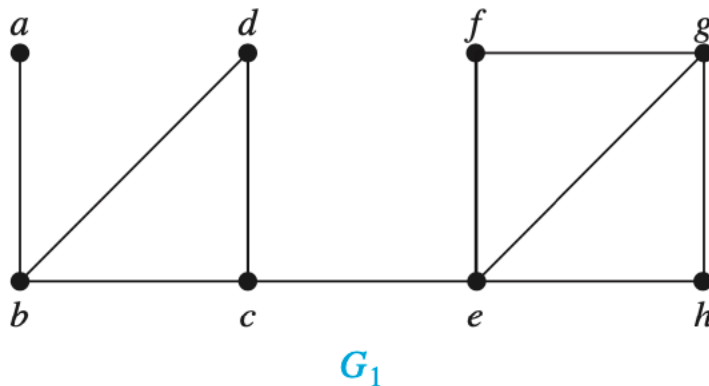
10.4.4 How Connected is a Graph?

Sometimes the removal from a graph of a vertex and all incident edges produces a subgraph with more connected components. Such vertices are called **cut vertices** 割点 (or **articulation points**). The removal of a cut vertex from a connected graph produces a subgraph that is not connected. Analogously, an edge whose removal produces a graph with more connected components than in the original graph is called a **cut edge** 割边 or **bridge**. Note that in a graph representing a computer network, a cut vertex and a cut edge represent an essential router and an essential link that cannot fail for all computers to be able to communicate.

EXAMPLE 7

Find the cut vertices and cut edges in the graph G_1 shown in Figure 4.

Solution: The cut vertices of G_1 are b , c , and e . The removal of one of these vertices (and its adjacent edges) disconnects the graph. The cut edges are $\{a, b\}$ and $\{c, e\}$. Removing either one of these edges disconnects G_1 .



VERTEX CONNECTIVITY

A subset V' of the vertex set V of $G=(V,E)$ is a **vertex cut** 点割集, or **separating set**, if $G-V'$ is disconnected.

We define the **vertex connectivity** 顶点连通度 of a noncomplete graph G , denoted by $\kappa(G)$, as the minimum number of vertices in a vertex cut.

When G is a complete graph, it has no vertex cuts, because removing any subset of its vertices and all incident edges still leaves a complete graph. Consequently, we cannot define $\kappa(G)$ as the minimum number of vertices in a vertex cut when G is complete. Instead, we set $\kappa(K_n) = n - 1$, the number of vertices needed to be removed to produce a graph with a single vertex.

EXAMPLE 8

Find the vertex connectivity for each of the graphs in Figure 4.

Solution: Each of the five graphs in Figure 4 is connected and has more than vertex, so each of these graphs has positive vertex connectivity. Because $G1$ is a connected graph with a cut vertex, as shown in Example 7, we know that $\kappa(G1) = 1$. Similarly, $\kappa(G2) = 1$, because c is a cut vertex of $G2$.

The reader should verify that $G3$ has no cut vertices, but that $\{b, g\}$ is a vertex cut. Hence, $\kappa(G3) = 2$. Similarly, because $G4$ has a vertex cut of size two, $\{c, f\}$, but no cut vertices. It follows that $\kappa(G4) = 2$. The reader can verify that $G5$ has no vertex cut of size two, but $\{b, c, f\}$ is a vertex cut of $G5$. Hence, $\kappa(G5) = 3$.

The larger $\kappa(G)$ is, the more connected we consider G to be.

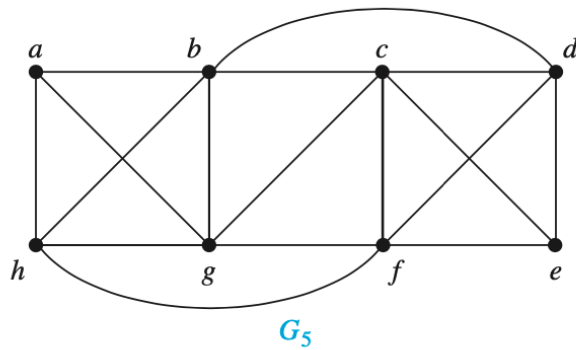
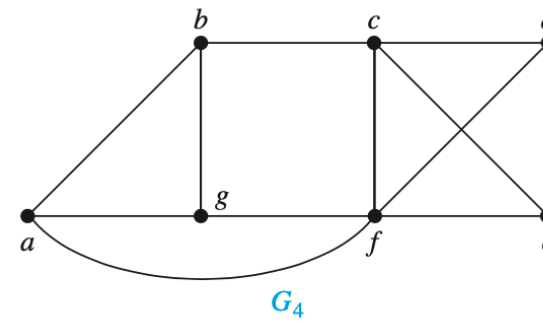
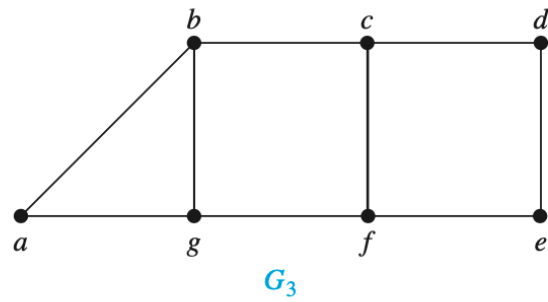
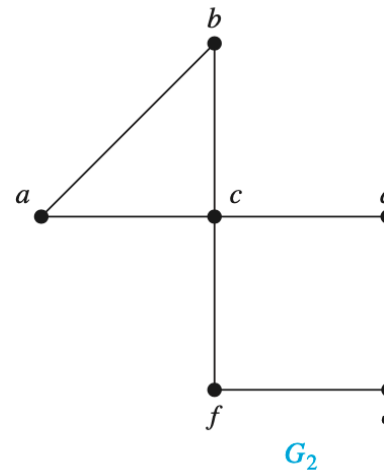
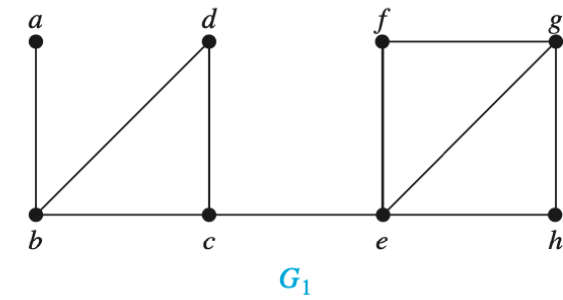


FIGURE 4 Some connected graphs.

EDGE CONNECTIVITY 边联通度 We can also measure the connectivity of a connected graph $G = (V, E)$ in terms of the minimum number of edges that we can remove to disconnect it. If a graph has a cut edge, then we need only remove it to disconnect G . If G does not have a cut edge, we look for the smallest set of edges that can be removed to disconnect it. A set of edges E' is called an **edge cut** 边割集 of G if the subgraph $G - E'$ is disconnected. The **edge connectivity** of a graph G , denoted by $\lambda(G)$, is the minimum number of edges in an edge cut of G .

EXAMPLE 9 Find the edge connectivity of each of the graphs in Figure 4.

Solution: Each of the five graphs in Figure 4 is connected and has more than one vertex, so we know that all of them have positive edge connectivity. As we saw in Example 7, G_1 has a cut edge, so $\lambda(G_1) = 1$.

The graph G_2 has no cut edges, as the reader should verify, but the removal of the two edges $\{a, b\}$ and $\{a, c\}$ disconnects it. Hence, $\lambda(G_2) = 2$. Similarly, $\lambda(G_3) = 2$, because G_3 has no cut edges, but the removal of the two edges $\{b, c\}$ and $\{f, g\}$ disconnects it.

The reader should verify that the removal of no two edges disconnects G_4 , but the removal of the three edges $\{b, c\}$, $\{a, f\}$, and $\{f, g\}$ disconnects it. Hence, $\lambda(G_4) = 3$. Finally, the reader should verify that $\lambda(G_5) = 3$, because the removal of any two of its edges does not disconnect it, but the removal of $\{a, b\}$, $\{a, g\}$, and $\{a, h\}$ does.

AN INEQUALITY FOR VERTEX CONNECTIVITY AND EDGE CONNECTIVITY

When $G = (V, E)$ is a noncomplete connected graph with at least three vertices, the minimum degree of a vertex of G is an upper bound for both the vertex connectivity of G and the edge connectivity of G .

That is, $\kappa(G) \leq \min_{v \in V} \deg(v)$ and $\lambda(G) \leq \min_{v \in V} \deg(v)$.

Section Summary⁴

1, Paths

Let n be a nonnegative integer and G an undirected graph. A *path* of *length* n from u to v in G is a sequence of n edges $e_1, \dots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \dots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \dots, n$, the endpoints x_{i-1} and x_i .

When the graph is *simple*, we denote this path by its vertex sequence $x_0, x_1, \dots, x_n$ (since listing the vertices uniquely determines the path).

A path or circuit is *simple* if it does not contain the same edge more than once.

2, Connectedness in Undirected Graphs

An undirected graph is called *connected* 连通的 if there is a path between every pair of vertices. An undirected graph that is not *connected* is called *disconnected*.

Definition: A *connected component* 联通分支 of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G .

Section Summary₄

3, Connectedness in Directed Graphs

A directed graph is *strongly connected* if there is a path from a to b and a path from b to a whenever a and b are vertices in the graph.

A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph

Definition: The subgraphs of a directed graph G that are strongly connected but not contained in larger strongly connected subgraphs, that is, the maximal strongly connected subgraphs, are called the *strongly connected components* 强联通分支 or *strong components* of G .

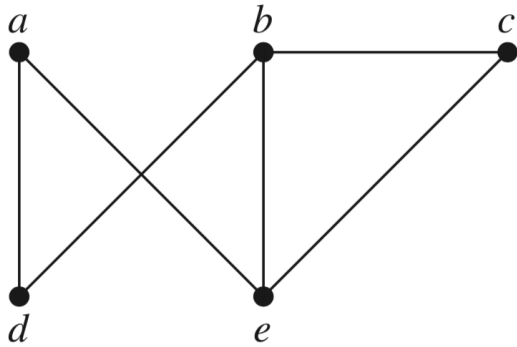
1. Does each of these lists of vertices form a path in the following graph? Which paths are simple? Which are circuits? What are the lengths of those that are paths?

a) a, e, b, c, b

b) a, e, a, d, b, c, a

c) e, b, a, d, b, e

d) c, b, d, a, e, c



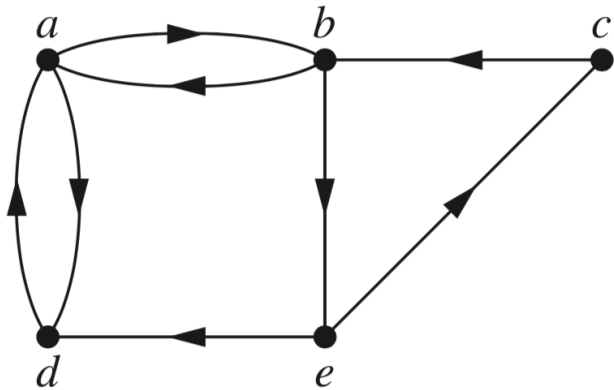
2. Does each of these lists of vertices form a path in the following graph? Which paths are simple? Which are circuits? What are the lengths of those that are paths?

a) a, b, e, c, b

b) a, d, a, d, a

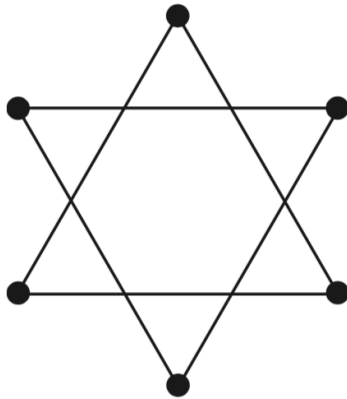
c) a, d, b, e, a

d) a, b, e, c, b, d, a



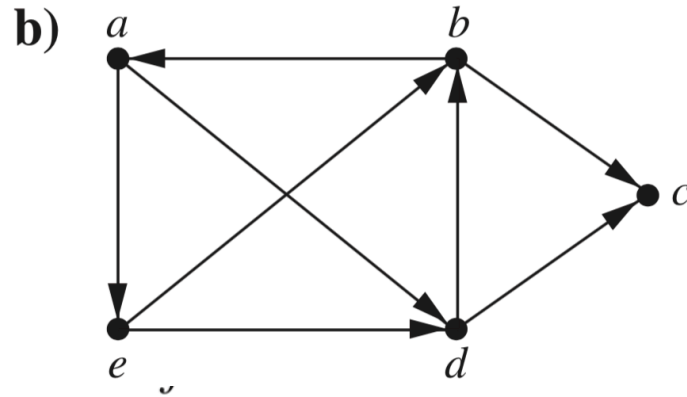
In Exercises 3–5 determine whether the given graph is connected.

5.

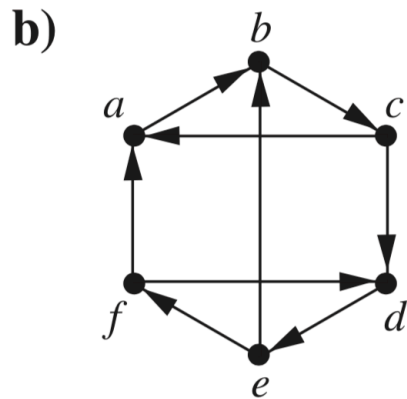


6. How many connected components does each of the graphs in Exercises 3–5 have? For each graph find each of its connected components.

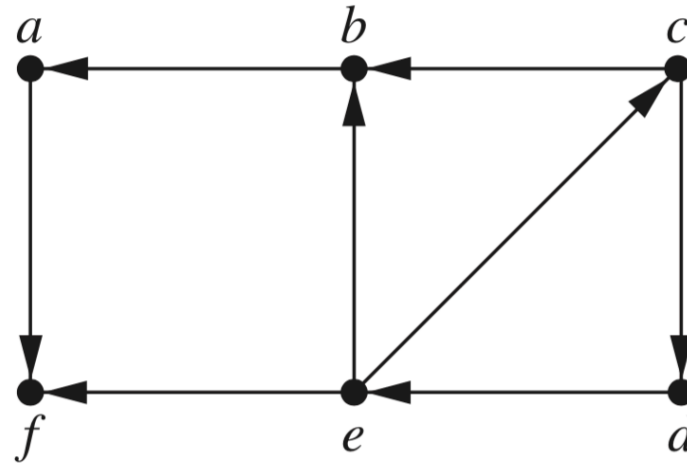
11. Determine whether each of these graphs is strongly connected and if not, whether it is weakly connected.



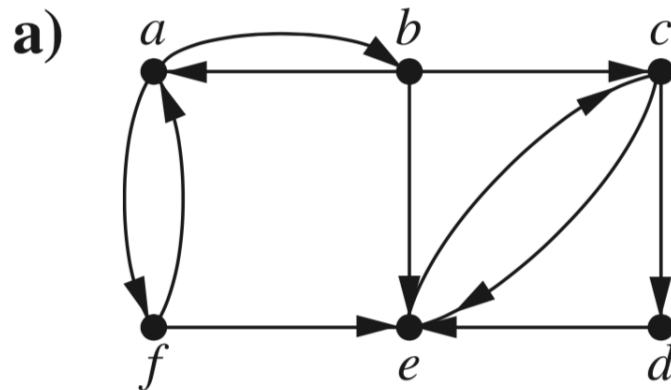
12. Determine whether each of these graphs is strongly connected and if not, whether it is weakly connected.



14. Find the strongly connected components of each of these graphs. **b)**



15. Find the strongly connected components of each of these graphs.



14 Therefore the strongly connected components are $\{a\}$, $\{b\}$, $\{c, d, e\}$, and $\{f\}$.

15 $\{a, b, f\}$, $\{c, d, e\}$

10.6 Shortest-Path Problems

10.6.2 A Shortest-Path Algorithm

ALGORITHM 1 Dijkstra's Algorithm.

```
procedure Dijkstra( $G$ : weighted connected simple graph, with  
    all weights positive)  
    {  $G$  has vertices  $a = v_0, v_1, \dots, v_n = z$  and lengths  $w(v_i, v_j)$   
      where  $w(v_i, v_j) = \infty$  if  $\{v_i, v_j\}$  is not an edge in  $G$  }  
    for  $i := 1$  to  $n$   
         $L(v_i) := \infty$   
     $L(a) := 0$   
     $S := \emptyset$   
    {the labels are now initialized so that the label of  $a$  is 0 and all  
      other labels are  $\infty$ , and  $S$  is the empty set}  
    while  $z \notin S$   
         $u :=$  a vertex not in  $S$  with  $L(u)$  minimal  
         $S := S \cup \{u\}$   
        for all vertices  $v$  not in  $S$   
            if  $L(u) + w(u, v) < L(v)$  then  $L(v) := L(u) + w(u, v)$   
                {this adds a vertex to  $S$  with minimal label and updates the  
                  labels of vertices not in  $S$ }  
    return  $L(z)$  { $L(z)$  = length of a shortest path from  $a$  to  $z$ }
```

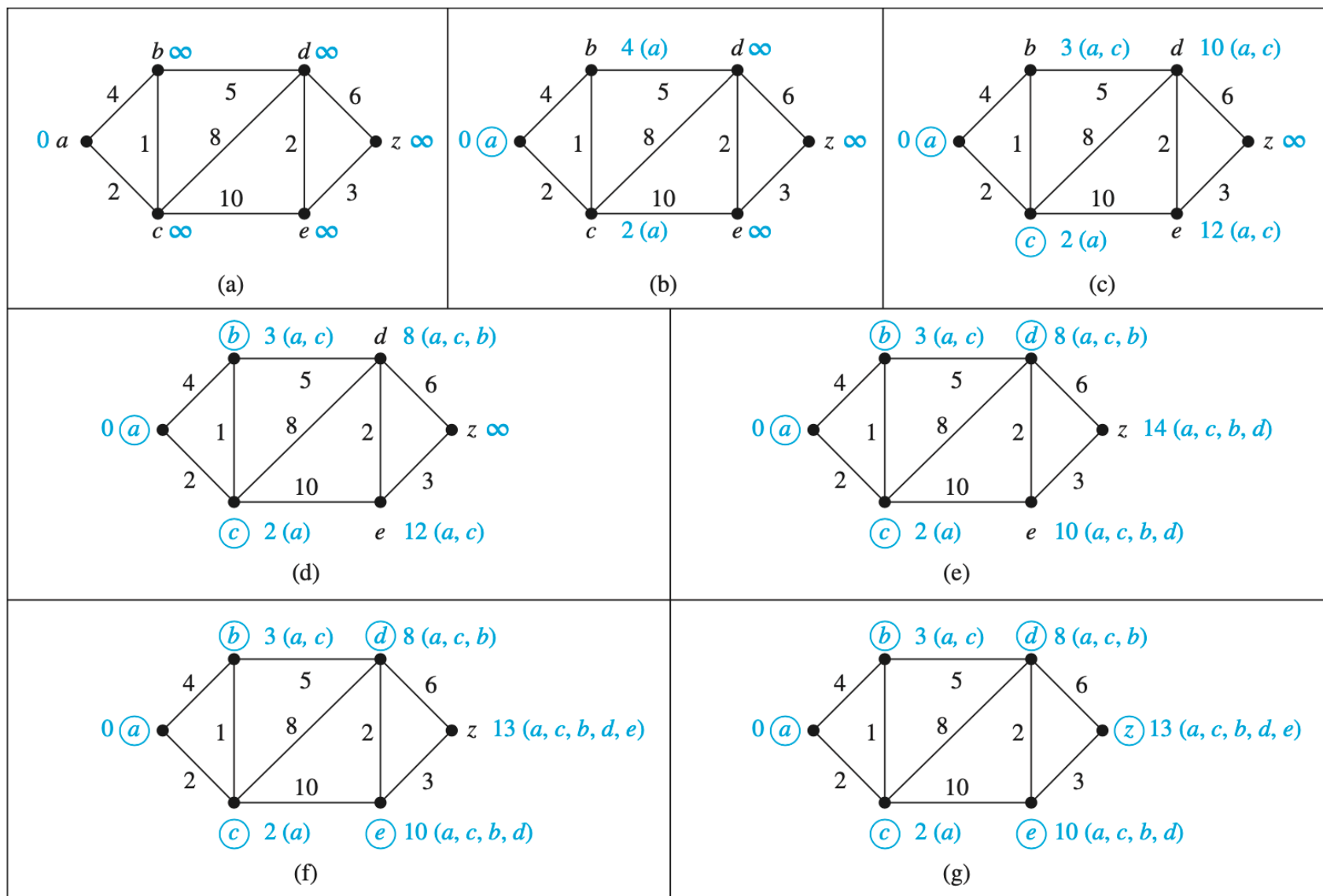


FIGURE 4 Using Dijkstra's algorithm to find a shortest path from a to z .

The shortest path

最短路径

最短路径问题

单源最短路径

BFS 算法（无权图）

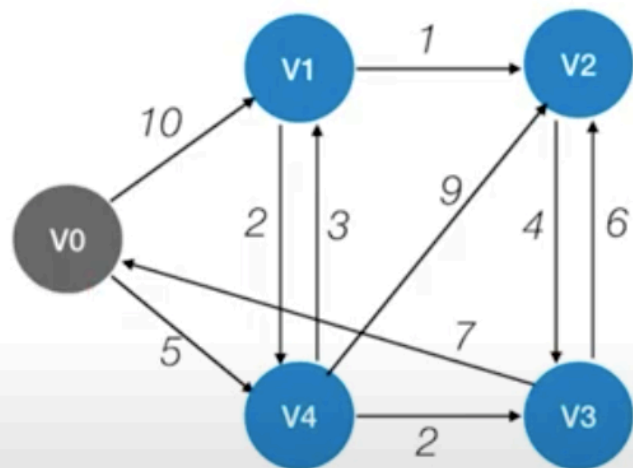
Dijkstra 算法（带权图、无权图）

各顶点间的最短路径

Floyd 算法（带权图、无权图）

Dijkstra算法

初始：从V₀开始，初始化三个数组信息如下 📌



标记各顶点是否
已找到最短路径

$final[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✗

最短路径长度

$dist[5]$

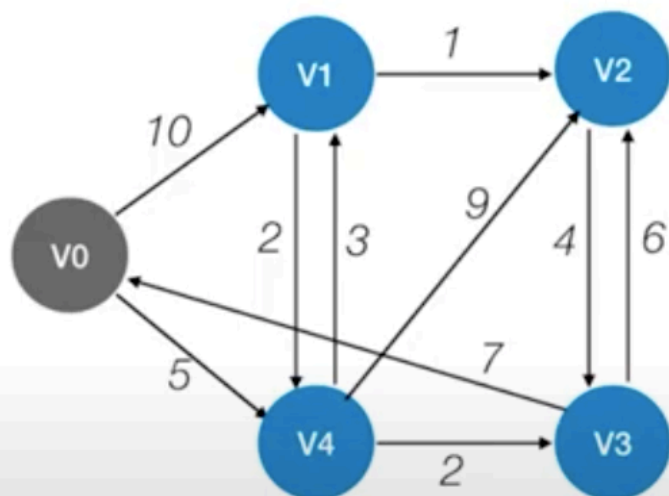
0	10	∞	∞	5
---	----	----------	----------	---

路径上的
前驱

$path[5]$

-1	0	-1	-1	0
----	---	----	----	---

Dijkstra算法



第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

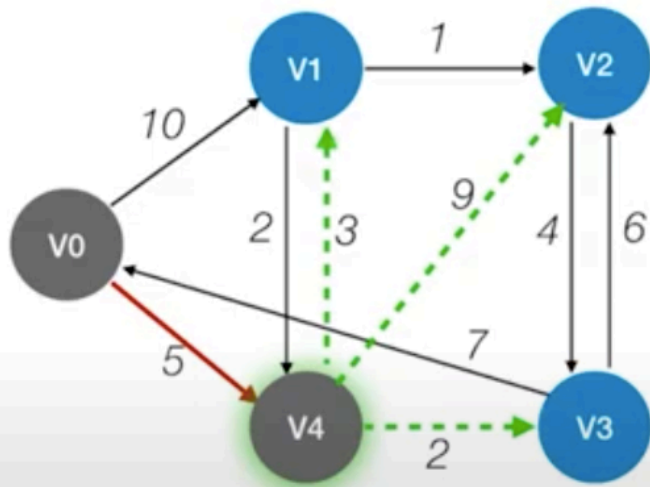
$path[5]$

v0	v1	v2	v3	v4
✓	✗	✗	✗	✗
0	10	∞	∞	5
-1	0	-1	-1	0



Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。



标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓
0	10	∞	∞	5
-1	0	-1	-1	0



检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

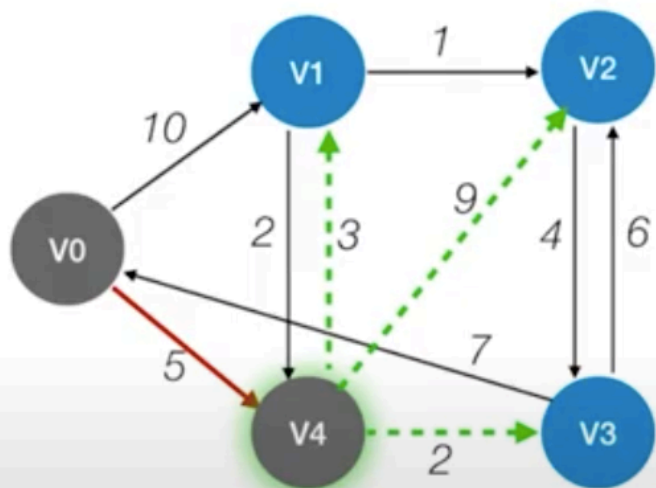
$dist[5]$

$path[5]$

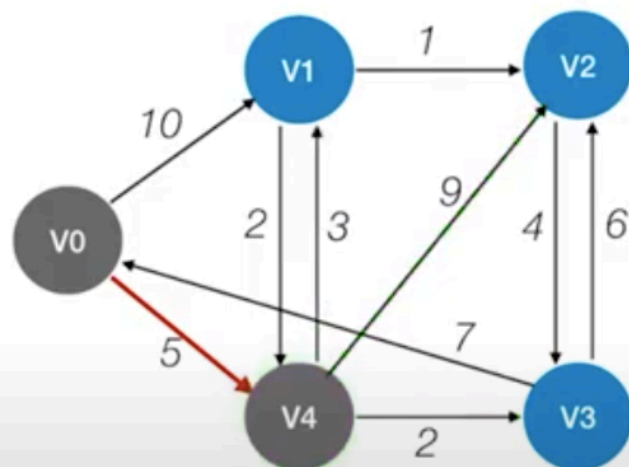
V0	V1	V2	V3	V4
✓	✗	✗	✗	✓
0	8	14	7	5
-1	4	4	4	0



检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息



Dijkstra算法



第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

$final[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓

最短路径长度

$dist[5]$

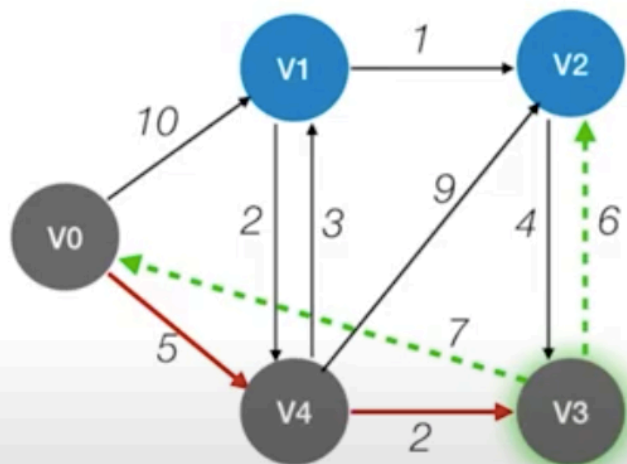
0	8	14	7	5
---	---	----	---	---

路径上的
前驱

$path[5]$

-1	4	4	4	0
----	---	---	---	---

Dijkstra算法



第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否已找到最短路径

$final[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✓	✓

最短路径长度

$dist[5]$

0	8	14	7	5
---	---	----	---	---

路径上的前驱

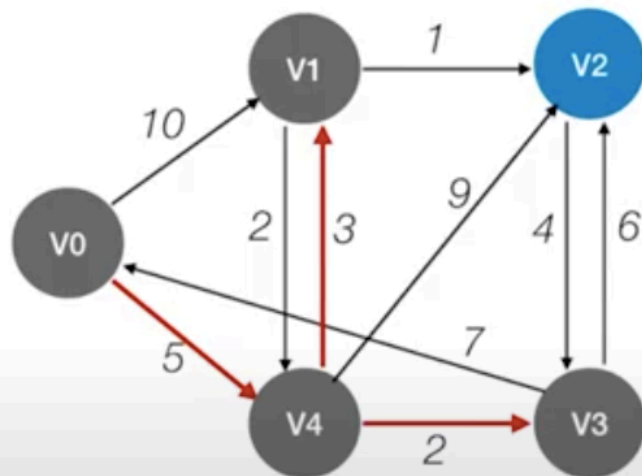
$path[5]$

-1	4	4	4	0
----	---	---	---	---

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息



Dijkstra算法



第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否已找到最短路径

最短路径长度

路径上的前驱

$final[5]$

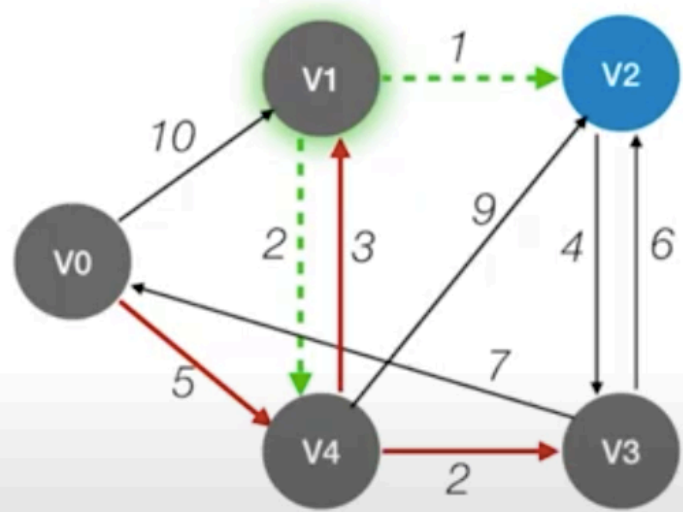
$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓
0	8	13	7	5
-1	4	3	4	0

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法



第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否已找到最短路径

最短路径长度

路径上的前驱

$final[5]$

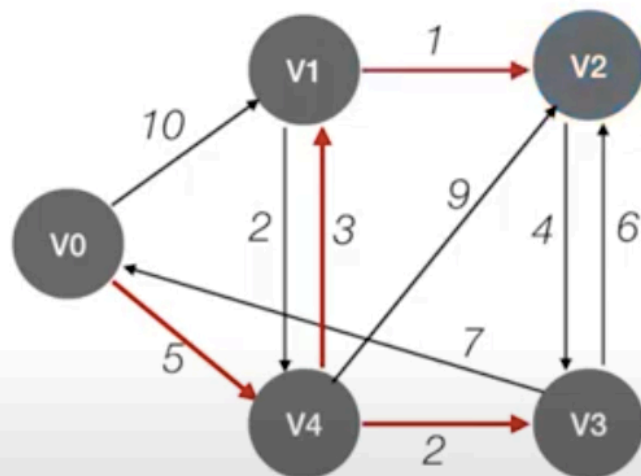
$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓
0	8	9	7	5
-1	4	1	4	0

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法



第4轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否已找到最短路径

最短路径长度

路径上的前驱

$final[5]$

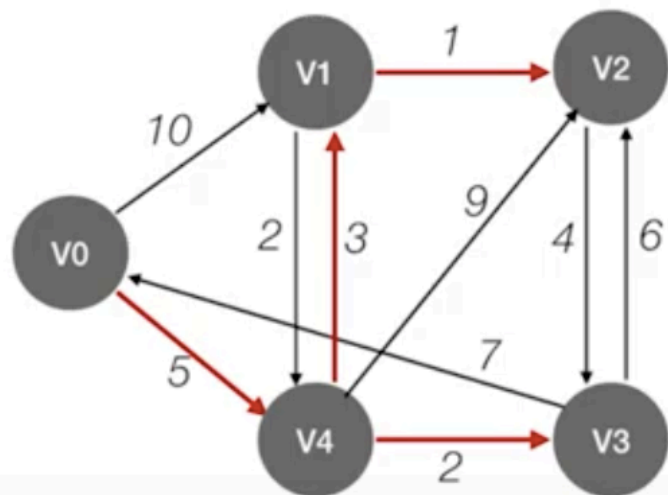
$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✓	✓	✓
0	8	9	7	5
-1	4	1	4	0

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

如何使用数组信息？



标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✓	✓	✓

0	8	9	7	5
---	---	---	---	---

-1	4	1	4	0
----	---	---	---	---

V0到V2 的最短(带权)路径长度为: $dist[2] = 9$

通过 $path[]$ 可知, V0到V2 的最短(带权)路径: $V2 \leftarrow V1 \leftarrow V4 \leftarrow V0$

- Reference:

<https://www.youtube.com/watch?v=yYR-XDU7C0Q>